Design And Analysis of Algorithms
Lecture 5: Shortest Paths.

February 8, 2018

This lecture focuses on computing shortest paths on a weighted graph G. Given a graph $G = (V, A)$, there are a number of path computations that one might want to complete. One many in this course will be familiar with is to ask for the shortest path between a fixed $s, t \in V$, called the "source" and "sink" respectively. When one assumes all edge weights are nonnegative, the standard algorithm for this task is Dijkstra's algorithm. In cases where edges might have negative weight, Dijkstra can fail; in this case the Bellman-Ford is a standard alternative. Bellman ford also computes the shortest path between s and *every* vertex t, a problem referred to as the "single source shortest path" (SSSP) problem. A related problem is the suggestively named "all pairs shortest path" (APSP) problem, where the goal is to compute the shortest path between each pair of vertices in G. These are progressively more difficult problems (in terms of their computational complexity): Dijsktra runs in $O(m + n \ln n)$ time; Bellman-Ford in $O(mn)$, and Johnson's algorithm in $O(mn \ln n)$, for arbitrary edge weights. Interestingly, Johnson's algorithm meets the runtime of n independent runs of Dijkstra, while also being valid for graphs with negative edge weights.

1 Dijkstra's Algorithm

We begin by refreshing our memory

Algorithm 1: Dijkstra's algorithm

```
Start at s ∈ V the source ;
Insert (s, 0) into priority queue P;
∀v ∈ V \ {s}, insert (v, ∞) into P;
while P is nonempty do
    (u_i, c_i) ←RemoveMin(P);
    Dist(u_i) = c_i;
    for all v : (u_i, v) ∈ A do
        DecreaseKey(P, v, c_i + w(u_i, v))
    end
end
```

This algorithm operates by processing each edge in turn, updating the cost of a path to a vertex in each round. There are therefore n inserts and deletes and $O(m)$ decrease key operations. Using a fibonacci heap [Fredman and Tarjan, 1987] implementation of a priority queue, this implies $O(m+n \ln n)$ total runtime of Dijkstra.

The primary downside of Dijkstra is that it will not notice negative cycles in a graph, since it never considers returning to a vertex it previously processed. Even graphs with negative edges but no negative cycles might cause Dijkstra to output an incorrect path. For graphs with negative edge weights, we must turn to some other algorithms.

2 The Bellman-Ford Algorithm

The first algorithm for SSSP which works with negative edge weights we will consider is Bellman-Ford. We need to extend the definition of the single-source shortest path problem to define what the algorithm should do when G has a negative cycle: this would make any path for which that negative cycle is reachable more expensive than a path with another iteration around the cycle. The standard requested behavior for a G with a negative cycle is for the algorithm to terminate and say that G has a negative cycle.

Here is the pseudocode for Bellman-Ford:

Algorithm 2: Bellman-Ford

```
Dist(s) = 0;
Dist(v) = ∞ for all v ∈ V, v ≠ s;
for i = 0 to n − 1 do
    for e ∈ A, e = (u, v) do
        if Dist(v) > Dist(u) + w_e then
            Dist(v) = Dist(u) + w_e;
        end
    end
end
for e ∈ E, e = (u, v) do
    if Dist(v) > Dist(u) + w_e then
        Return "negative cycle";
    end
end
```

The following lemma is the crux of the proof of Bellman-Ford's correctness.

Lemma 1. *At the end of iteration i, $\texttt{Dist}(v)$ contains the cost of the cheapest $\leq i$ hop path from s to v.*

Proof. By induction on i. □

The remaining argument of correctness needs to show that the algorithm has the correct behavior when G has a negative cycle. If G has no negative cycle, any min-cost path has length at most $n-1$, meaning Lemma 1 implies in this case that Bellman-Ford prior to the final for-loop will compute the correct distances to vertices from s. The final for-loop checks if an additional hop would reduce the cost of any path; if it does, that implies there is a negative cycle in G (and so the algorithm's behavior is correct); moreover, if there is a negative-weight cycle in G, there will be some path which is made cheaper by having an n-th hop. So, the algorithm has the correct behavior.

The runtime of the algorithm is $O(mn)$, worse than Dijkstra because of the need to worry about negative cycles.

3 All Pairs Shortest Paths

In this section, we discuss the all-pairs shortest path problem, allowing G to have negative edge weights. To run n iterations of Dijkstra independently (converting the SSSP algorithm into an APSP) would take $O(mn + n^2 \ln n)$ time; we will present Johnson's algorithm which matches this runtime but (unlike Dijkstra) works on general graphs. Pettie [2002] improves upon this runtime to $O(mn + n^2 \ln \ln n)$, but we won't present that algorithm here.

The idea behind this algorithm is to find some new weight function $\hat{w}$ for which the shortest paths don't change from G to $\hat{G}$, such that $\hat{w}(v) \geq 0$. This shouldn't be possible if there are negative cycles, since any path will now have nonnegative weight. If we find such a weighting, we can then run n instances of Dijsktra on $\hat{G}$ and be guaranteed its correctness and runtime for free.

Definition 1 (Valid potential)**.** *A valid potential for G is a mapping $\phi : V \to \mathbb{R}$ such that for all $e = (u, v) \in A$, $\phi(v) - \phi(u) + w(e) \geq 0$.*

Notice that such a potential won't exist for any G with negative cycles, because it would mean $\phi(v) - \phi(u) \geq -w(e)$ and summing up around the cycle would mean $0 \geq \sum_{e \in C} -w(e) > 0$, a contradiction.

Now, given a valid potential ϕ, define $\hat{w}(u, v) = w(u, v) + \phi(v) - \phi(u)$. This will have nonnegative edge weights, since ϕ is a valid potential. Moreover, any path P from u to v will have cost $w(P) + \phi(u) - \phi(v)$, so the ordering of paths by cost from u to v won't change.

All that remains then is to construct ϕ for G relatively quickly. The following lemma will guide this construction.

Lemma 2. *Suppose there is some $s \in V$ from which all other $v \in V$ are reachable. Then, $\phi(v) = \texttt{Dist}(u, v)$ is a valid potential for G if G has no negative cycles.*

Proof. Given any $e = (u, v) \in A$, we need to show that $\phi(u) - \phi(v) + w_e \geq 0$. This is equivalent to showing that $\texttt{Dist}(s, u) - \texttt{Dist}(s, v) + w_e \geq 0$: but this must be the case since a path from s to u then taking edge e cannot be shorter than the shortest path from s to v in a graph with no negative cycles. □

So, for G with a "source" of this form, a single run of Bellman-Ford can compute *phi*. If G doesn't have a source vertex; we can simply add one with zero-cost edges to each other vertex and do the same.

In total the runtime of this will be $O(mn)$ for the run of Bellman-Ford and $O(mn + n^2 \ln n)$ for the n iterations of Dijkstra.

4 Floyd-Warshall's APSP algorithm and Matrix Multiplication

We now briefly outline another algorithm for the APSP: Floyd-Warshall. This algorithm is slower than Johnson's algorithm (it will run in $O(n^3)$ time) but it's simple and highly parallelizeable. It will also be useful in showing a connection between APSP and several fundamental calculations on matrices.

Algorithm 3: Floyd-Warshall's APSP

```
d(x, y) = ∞ for all x ≠ y ∈ V;
d(x, x) = 0 for all x ∈ V;
for z ∈ V do
    for x, y ∈ V do
        d(x, y) ← min(d(x, y), d(x, z) + d(zy, y))
    end
end
if there is some v ∈ V for which d(v, v) < 0 then
    Return "Negative cycle";
end
```

Its correctness proof follows along the same lines as Bellman-ford in that each iteration adds a hop to the set of paths it considers for minimum cost.

4.1 Matrix Multiplication, Sum-Product and Min-Sum operations

Given two $m \times n$ real-valued matrices $A, B \in \mathbb{R}^{n \times n}$, matrix multiplication is defined to be

$$(A \times B)_{i,j} = \sum_{k \in [n]} A_{ik} B_{kj}$$

which, when written this way, is more obviously asking for n sums of products of the entries of A and B. The problem of computing sum-products is fundamental to much of computing. So fundamental, in fact,

that we use $O(n^\omega)$ to refer to the optimal runtime for this problem, which has not been determined exactly. The brute-force algorithm takes $O(n^3)$ time, and the current best-known $O(n^{2.3728639})$ due to Le Gall [2014], a slight improvement over $O(n^{2.3729})$ proven by Williams [2012].

The min-sum operator $\circledcirc$ A related operation on $n \times n$-dimensional matrices is the min-sum operation, which produces a matrix defined as

$$(A \circledcirc B)_{i,j} = \min_{k \in [n]} A_{ik} + B_{kj}.$$

This operator is particularly useful for the problems we investigated in this class if we look at the adjacency matrix A of a graph G, defined as

$$A_{ij} = \begin{cases} w_e & \text{if } (i,j) \in E \\ 0 & \text{if } i = j \\ \infty & \text{if } i \neq j, (i,j) \notin E \end{cases}.$$

Then, $A \circledcirc A$ computes the least expensive hop 1 or 2 path between each pair of vertices, and $A^{\circledcirc t}$ the least expensive path of at most t hops. This operator calculates the inner for-loop of the Floyd-Warshall algorithm; so, if it's possible to compute it quickly, it might allow us to speed up an implementation thereof. In other words, , $A^{\circledcirc n}$ does the same computation as Floyd-Warshall, and any way to compute $A^{\circledcirc n}$ in faster that $O(n^3)$ time would speed up Floyd-Warshall.

How quickly can we compute $A^{\circledcirc n}$, then? One simple upper bound is of $n \times MSP(n)$, where $MSP(n)$ is the time to compute one min-sum product of two $n \times n$-dimensional matrices. A better one notices that $A^{\circledcirc n} = (A^{\circledcirc n/2}) \circledcirc (A^{\circledcirc n/2})$ (assuming n is even), so one can actually compute $A^{\circledcirc n}$ with only logarithmically many min-sum products. So, we now have an upper bound of $O(\log n) \cdot MSP(n)$. A brute-force algorithm can compute min-sums in $O(n^3)$ time (just like matrix products); one might then ask if we have some similar tools for speeding up min-sum products that we used for sum-products.

Unfortunately, most of the tricks we know for speeding up matrix multiplication (e.g., Strassen's algorithm) rely upon the fact that those operations form a ring, while min-sums only form a semiring. In fact, we don't know how to compute min-sums in $O(n^{3-\epsilon}$ for any constant ϵ in the worst case. Ryan Williams [Williams, 2014] has the current optimal runtime for this problem, of $O\left(\frac{n^3}{2^{\sqrt{\ln n}}}\right)$, a slight improvement over $O(\frac{n^3}{\log n})$ the previous best-known runtime for min-sum product [Fredman, 1976].

References

Don Coppersmith and Shmuel Winograd. Matrix multiplication via arithmetic progressions. In *Proceedings of the nineteenth annual ACM symposium on Theory of computing*, pages 1–6. ACM, 1987.

Michael L Fredman. New bounds on the complexity of the shortest path problem. *SIAM Journal on Computing*, 5(1):83–89, 1976.

Michael L Fredman and Robert Endre Tarjan. Fibonacci heaps and their uses in improved network optimization algorithms. *Journal of the ACM (JACM)*, 34(3):596–615, 1987.

François Le Gall. Powers of tensors and fast matrix multiplication. In *Proceedings of the 39th international symposium on symbolic and algebraic computation*, pages 296–303. ACM, 2014.

Seth Pettie. A faster all-pairs shortest path algorithm for real-weighted sparse graphs. In *International Colloquium on Automata, Languages, and Programming*, pages 85–97. Springer, 2002.

Raimund Seidel. On the all-pairs-shortest-path problem in unweighted undirected graphs. *Journal of computer and system sciences*, 51(3):400–403, 1995.

Avi Shoshan and Uri Zwick. All pairs shortest paths in undirected graphs with integer weights. In *Foundations of Computer Science, 1999. 40th Annual Symposium on*, pages 605–614. IEEE, 1999.

Ryan Williams. Faster all-pairs shortest paths via circuit complexity. In *Proceedings of the forty-sixth annual ACM symposium on Theory of computing*, pages 664–673. ACM, 2014.

Virginia Vassilevska Williams. Multiplying matrices faster than coppersmith-winograd. In *Proceedings of the forty-fourth annual ACM symposium on Theory of computing*, pages 887–898. ACM, 2012.

Uri Zwick. All pairs shortest paths using bridging sets and rectangular matrix multiplication. *Journal of the ACM (JACM)*, 49(3):289–317, 2002.